



Code Evolution Graphs: Understanding Large Language Model Driven Design of Algorithms

Niki van Stein

n.van.stein@liacs.leidenuniv.nl
LIACS, Leiden University
Leiden, Netherlands

Lars Kotthoff

larsko@uwyo.edu
University of Wyoming
Laramie, United States

Anna V. Kononova

a.kononova@liacs.leidenuniv.nl
LIACS, Leiden University
Leiden, Netherlands

Thomas Bäck

t.h.w.baeck@liacs.leidenuniv.nl
LIACS, Leiden University
Leiden, Netherlands

Abstract

Large Language Models (LLMs) have demonstrated great promise in generating code, especially when used inside an evolutionary computation framework to iteratively optimize the generated algorithms. However, in some cases they fail to generate competitive algorithms or the code optimization stalls, and we are left with no recourse because of a lack of understanding of the generation process and generated codes. We present a novel approach to mitigate this problem by enabling users to analyze the generated codes inside the evolutionary process and how they evolve over repeated prompting of the LLM. We show results for three benchmark problem classes and demonstrate novel insights. In particular, LLMs tend to generate more complex code with repeated prompting, but additional complexity can hurt algorithmic performance in some cases. Different LLMs have different coding “styles” and generated code tends to be dissimilar to other LLMs. These two findings suggest that using different LLMs inside the code evolution frameworks might produce higher performing code than using only one LLM.

CCS Concepts

• **Theory of computation** → **Design and analysis of algorithms**; **Design and analysis of algorithms**; **Bio-inspired optimization**; • **Human-centered computing** → *Visualization techniques*; • **Computing methodologies** → **Heuristic function construction**.

ACM Reference Format:

Niki van Stein, Anna V. Kononova, Lars Kotthoff, and Thomas Bäck. 2025. Code Evolution Graphs: Understanding Large Language Model Driven Design of Algorithms. In *Genetic and Evolutionary Computation Conference (GECCO '25)*, July 14–18, 2025, Málaga, Spain. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3712256.3726328>

1 Introduction

The automated generation and optimization of algorithms has become a critical area of research in evolutionary computation and

artificial intelligence. Traditional approaches to designing optimization algorithms often rely on expert knowledge, which can be time-consuming and prone to human bias. Recently, Large Language Models (LLMs) such as GPT-4 [12] have demonstrated significant potential for automating this process by synthesizing and refining algorithms based on natural language prompts and feedback [4, 14, 18, 24].

One of the key challenges in automated algorithm design with LLMs is the limited direct control we have over the algorithm generation, mutation and crossover. This leads to a limited understanding of how the evolutionary search through code space is performing and how we can improve these algorithms in a systematic way. To address this, we propose the concept of *Code Evolution Graphs* (CEGs), which integrate static code analysis, graph-based representations, and complexity metrics to provide insights into optimization dynamics and the resulting algorithmic behaviors of code generation and optimization frameworks. By utilizing Abstract Syntax Trees (ASTs) as a basis, CEGs enable the extraction of static code features, offering a comprehensive toolbox for understanding the evolution of algorithms over successive generations.

In this paper, we focus on analyzing the structural and complexity characteristics of algorithms generated by the Large Language Model Evolutionary Algorithm (LLaMEA) [18], LLaMEA with in-the-loop hyper-parameter optimization (LLaMEA-HPO) [20] and Evolution of Heuristics (EoH) [4], across three benchmark tasks: Black-Box Optimization (BBO), Online Bin Packing (OBP), and the Traveling Salesperson Problem (TSP). We aim to answer key questions about the nature of the algorithms produced, such as their structural modularity, logical complexity, and scalability and how these features evolve during the (meta)optimization runs.

The main contributions of this work are as follows:

- We introduce Code Evolution Graphs, a novel methodology that combines graph-based representations and complexity analysis to evaluate the evolution of code generated by LLMs and evolutionary frameworks.
- We systematically extract and analyze 20 structural and complexity features from ASTs, including cyclomatic complexity, token counts, clustering coefficients, and entropy measures, to uncover patterns in algorithmic design.
- We provide a comparative analysis of LLaMEA and EoH across diverse optimization tasks, revealing the strengths and limitations of LLM-driven algorithm generation.



This work is licensed under a Creative Commons Attribution 4.0 International License. *GECCO '25*, July 14–18, 2025, Málaga, Spain
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1465-8/2025/07
<https://doi.org/10.1145/3712256.3726328>

- We visualize the evolution of code with PCA and t-SNE to uncover high-level trends and structural shifts that provide novel insights into the optimization process.

Recent advancements in LLM-based search frameworks, such as Funsearch [14], LLaMEA [18] and EoH [4], have demonstrated the feasibility of using natural language processing models to generate novel algorithms whose performance rivals that of state-of-the-art heuristic methods. Funsearch was among the first methods that showed that with evolutionary search techniques LLMs could generate novel and well performing heuristics; however, Funsearch required a few million evaluations to do so at that time. LLaMEA leverages an iterative process of algorithm generation, evaluation, and refinement, guided by fitness feedback and error correction, to evolve complex optimization strategies, originally proposed to solve black-box optimization tasks [18]. In contrast, EoH focuses on the evolution of smaller heuristic functions through mutation and crossover operations [4], originally aimed at solving combinatorial problems. Both approaches have shown promising results on benchmark tasks, but we lack a detailed understanding of the structural and functional properties of the generated code and the evolution thereof.

The introduction of Code Evolution Graphs addresses this gap by providing a unified framework for analyzing a wide range of attributes of the codes generated over the optimization run. By leveraging AST-based feature extraction and graph-theoretic metrics, we aim to gain deeper insights into the design principles underlying LLM-generated algorithms and their impact on performance.

2 Related Work

The automated generation and optimization of algorithms has been a topic of discussion in the optimization community since at least the early 2010s, when [6] introduced the concept of *Programming by Optimization*. To avoid premature commitment to design choices in the algorithm implementations, developers were encouraged instead to maintain a selection of alternatives, forming a parameterized combinatorial algorithm design space that can be searched by a meta-optimizer for better empirical performance over representative problems. The concept has been subsequently practically extended to allow continuous/mixed design spaces [17] and introduced to numerical black-box optimization context as *modular algorithms* [23], where a meta-optimizer searches over algorithms with a defined structure and a known range of options for all modules. These developments have subsequently been recognized as part of the broader task of *automated algorithm design* (AAD) for optimisation [22]. With the advent of LLM-based approaches, we argue here for the extension of AAD to include the automated generation and optimization of algorithms without prior specification of algorithm structure. Thus, in the remainder of this paper, we focus exclusively on AAD for the generation and optimization of meta-heuristic for optimization tasks, referring to them as AAD.

Analyzing the behavior of optimization algorithms has been a central theme in black-box optimization research, with various approaches proposed to gain insights into algorithm performance, search dynamics, and problem landscapes. While this paper represents the first effort to analyze code evolution in the AAD domain,

we draw upon related work in algorithm trajectory analysis, landscape modeling, and static code analysis to ground our approach.

2.1 Analysis of Optimization Algorithm Behavior

Search Trajectory Networks (STNs) have been employed to visualize and understand the movement of algorithms in the search space [10]. STNs represent the trajectory of an algorithm as a directed graph, where nodes correspond to search locations and edges denote transitions between them. This approach has been particularly useful for identifying common pathways and termination points in the search process. However, STNs typically do not capture temporal information, such as how long an algorithm remains in a particular region. Attractor networks, introduced in [16], address this limitation by focusing on regions of stagnation, or “attractors” where an algorithm stalls for a predefined number of evaluations. These attractor networks provide a coarser-grained view of algorithm dynamics, revealing insights into stalling behavior that are not evident in traditional STNs.

Another form of attractor analysis for continuous optimization, is by using local optima networks (LONs) [11]. LONs represent the connectivity and distribution of local optima in a search space, providing insights into the basins of attraction. However, LONs often rely on local search methods for their construction, limiting their generalization to other algorithms.

Exploratory Landscape Analysis (ELA) provides another lens for understanding optimization problems by characterizing the fitness landscape using features such as ruggedness, separability, and global structure [8]. These features are essential for predicting algorithm performance and tailoring strategies to specific problem characteristics. While ELA focuses on the BBO problem landscape, our method focuses on the AAD problem landscape, and more specifically the structural properties of the algorithms found during the search, extracting features from the code to understand how they evolve and adapt to various tasks.

While the above methods are applied to gain insights into algorithm-problem interactions, they cannot be (straightforwardly) applied to AAD tasks. The main challenge is that “individuals” in the AAD task are algorithms, with no clearly defined representation that can be used to compute similarities. While some work represents algorithms as vectors based on pre-defined modules of the algorithms [21], this approach is not possible if the code is generated and does not adhere to any pre-defined modular structure.

2.2 Static Code Analysis

Static code analysis, a well-established technique in software engineering, has been instrumental in assessing the complexity, maintainability, and quality of source code [9]. Metrics such as cyclomatic complexity and code token analysis provide quantitative measures of code structure. When we speak of ‘tokens’ in this work, we mean lexical tokens of the code, indicating total code size.

The work by Pulatov et al. [13] introduces an approach to improving algorithm selection by analyzing structural features of algorithms, moving beyond traditional “black-box” methods that rely solely on performance observations. By using static code analysis and AST features such as cyclomatic complexity and clustering

coefficients, the authors demonstrate that incorporating algorithm features into the selection process can lead to performance improvements. This perspective aligns with our goal of understanding the evolution of LLM-generated algorithms by analyzing their structural and complexity characteristics, as captured by AST-based metrics. Inspired by this, our work applies static code analysis to evaluate the structural and complexity features of algorithms generated in the AAD domain to provide a comprehensive understanding of code evolution.

Visualization techniques for changes in code over time were proposed in the past to mainly visualize changes in repositories or software libraries. In [15] for example, the authors introduce the Code Flows methodology, a visualization technique for understanding structural changes in source code across multiple versions. Instead of visualizing a complete repository of code based on a Git or change log, we propose a visualization and analysis methodology for evolving code (single-file algorithms or even single functions) generated by LLMs in an AAD setting.

Despite these advancements, no prior work has systematically analyzed the structural and complexity characteristics of algorithms generated in the AAD domain. This gap motivates our study, which combines techniques from STNs, attractor analysis, and static code analysis to offer a novel perspective on algorithm design and evolution. By extracting and analyzing a diverse set of features from LLM-generated code, we aim to uncover patterns and principles that underlie successful algorithm generation, aiming to contribute to a deeper understanding of the AAD landscape.

3 Methodology

In this section, we introduce the methodological framework for analyzing the evolution of code generated during the optimization process using Large Language Models and evolutionary frameworks. Our approach focuses on extracting meaningful insights into the properties of the generated algorithms that affect performance. To achieve this, we leverage Abstract Syntax Trees as a foundational representation of the code, from which we derive a comprehensive set of metrics and features. These include structural properties, complexity measures, and graph-based representations that collectively characterize the behavior of the algorithms.

We begin by describing the *Abstract Syntax Tree Features* (Section 3.1), which capture the syntactic structure of the code and enable the computation of graph-theoretic metrics. Next, we introduce the *Code Complexity Features* (Section 3.2) we use, focusing on cyclomatic complexity, token counts, and parameter counts to quantify the logical and structural complexity of the generated algorithms. Finally, we formally define the concept of *Code Evolution Graphs* (Section 3.3), a novel graph-based representation that models the lineage and transformations of code over successive generations in the optimization process.

3.1 Abstract Syntax Tree Features

Abstract Syntax Trees serve as a foundational representation of source code structure, enabling the extraction of syntactic features for code analysis. ASTs can be seen as directed acyclic graphs (DAGs), where nodes represent syntactic elements (e.g., loops, conditionals), and edges capture their hierarchical relationships. This

facilitates the computation of graph-theoretic metrics and other structural properties.

To characterize ASTs, we extract the following features:

Structural Properties Metrics such as node count, edge count, and depth distribution quantify code complexity.

Graph Centrality Eigenvector centrality highlights critical nodes influencing graph connectivity.

Clustering Coefficients Measures of node clustering reveal the modularity of the code structure.

Transitivity and Assortativity These metrics reflect code cohesiveness and structural correlations.

Entropy Measures Degree and depth entropy quantify the diversity and balance of the AST structure.

This comprehensive set of metrics enables a comparative analysis of algorithmic structures and diversity and serves as a basis for visualizing code evolution.

3.2 Code Complexity Features

Understanding the complexity of generated code is essential for evaluating its scalability, maintainability, and computational efficiency. To this end, we leverage a range of static analysis metrics to quantify the complexity of code structures produced during the optimization process. These metrics are particularly important for assessing the trade-offs between algorithmic sophistication and runtime performance.

The following key complexity features are extracted:

Cyclomatic Complexity This metric quantifies the number of linearly independent paths through the code, providing a measure of its logical complexity [3].

Token Count The total count of lexical tokens indicates the overall size of the code and its potential for human readability or interpretability.

Parameter Count The number of parameters in functions and methods reflects the modularity and configurability of the code.

Function-Level Aggregates Per-function averages and totals for cyclomatic complexity, token count, and parameter count are computed to understand variations within the codebase.

Depth and Nesting Metrics The maximum and average levels of nesting provide insights into structural depth and potential readability challenges.

These complexity measures are computed with the Lizard [7] tool for static code analysis and further enriched by custom AST-based feature extraction. The total resulting code features we extract and use are listed in Table 1.

3.3 Code Evolution Graphs

We define a *Code Evolution Graph (CEG)* as a directed graph $G = (V, E)$, where:

V is the set of nodes, each corresponding to an algorithmic instance represented by a vector of extracted features from its AST and associated metadata. The nodes $v_i \in V$ are defined by:

$$v_i = (f_i, x_i, m_i)$$

where:

Table 1: Features Extracted for Code Analysis

Feature Name	Explanation
Node Count	Total number of nodes in the AST, reflecting the size of the code structure.
Edge Count	Total number of edges in the AST, representing the connections between syntactic elements.
Edge Density	Ratio of edges to nodes in the AST, providing a measure of graph connectivity.
Min. Max. Mean Var. Degree	Statistical features about the degrees of the nodes in the AST.
Degree Entropy	Entropy of node degrees, capturing variability in connectivity.
Assortativity	Degree assortativity of the AST, measuring structural correlations between connected nodes.
Min. Max. Mean. Depth	Features regarding the depth of the AST, indicating the level of nested constructs.
Depth Entropy	Entropy of node depths, quantifying structural balance in the AST.
Min. Max. Mean Var. Clustering	Clustering coefficient features of the AST nodes, indicating the modularity of code.
Transitivity	Global measure of clustering in the AST, representing code cohesion.
Diameter	Longest shortest path in the AST, representing the maximum distance between two nodes.
Radius	Shortest maximum distance from any node to all other nodes, indicating graph compactness.
Mean Eccentricity	Mean eccentricity of nodes (eccentricity is the longest distance from a node to any other node).
Average Shortest Path	Mean shortest path length in the AST, representing structural compactness.
Cyclomatic Complexity	Logical total and mean code complexity, representing the number of independent execution paths.
Token Count	Total and average number of tokens in the code, reflecting its size and verbosity.
Parameter Count	Number of parameters in functions and average number per function, reflecting configurability.

- $f_i \in [0, 1]$ is the normalized performance of the algorithm on the respective benchmark;
- $\mathbf{x}_i \in \mathbb{R}^d$ is the d -dimensional vector of standardized AST features, such as cyclomatic complexity, token count, parameter count, and other extracted metrics;
- m_i is the *metadata* associated with the algorithm, including identifiers (e.g., id and name), evaluation index within the code optimization framework (such as LLaMEA) run, and parent IDs.

$E \subseteq V \times V$ is the set of directed edges, where an edge $e = (v_i, v_j)$ exists if algorithm v_j was generated as a descendant of v_i during an optimization process. Parent-child relationships are determined from the evolutionary lineage.

Visualization: To analyze the evolution of algorithms over the AAD evaluations, CEGs are projected into lower-dimensional spaces using dimensionality reduction techniques, in this case using Principal Component Analysis (PCA) [1] and t-distributed Stochastic Neighbor Embedding (t-SNE) [2]. Nodes are plotted in the reduced space, with node sizes proportional to their frequency as parents in the evolutionary lineage.

Purpose: Code Evolution Graphs provide a structured representation of algorithm evolution, enabling the analysis of structural changes in the code, fitness improvements, and lineage over successive generations.

4 Analysis of Different Code Evolution Strategies

To evaluate the effectiveness of the Code Evolution Graphs and gain insights into algorithm design and evolution, we consider three benchmark tasks: Black-Box Optimization (BBO), Online Bin Packing (OBP), and the Traveling Salesperson Problem (TSP). Each benchmark represents a distinct optimization challenge, enabling a comprehensive analysis of the generated algorithms' adaptability, scalability, and structural complexity in the AAD setting.

For the visualizations in this work, we split the AST and complexity features and use the AST features as input for the PCA transformation. We choose to look at complexity separately as we saw a strong linear correlation of the different complexity features with each other. To analyse the complexity of the generated algorithms, we mainly look at code token counts, as the cyclomatic complexity and number of function parameters are highly influenced by the way the evolutionary frameworks are organized. For example, for the LLaMEA-HPO algorithm, the prompt asks explicitly to add any tunable parameter to the function prototype, causing a much higher number of parameters by construction.

4.1 Black-Box Optimization

The Black-Box Optimization benchmark suite consists of a set of continuous, noiseless functions that challenge the optimization capabilities of metaheuristic algorithms. The objective is to minimize a given function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, where no information about the function's structure or derivatives are available. Instead, algorithms must iteratively query the function and refine their search based solely on previously evaluated solutions.

In the AAD task, the goal is to automatically generate and optimize metaheuristics capable of solving a diverse set of BBO functions. The algorithms evolved by frameworks such as LLaMEA and EoH are evaluated on their ability to balance exploration and exploitation across the black-box optimization landscape by evaluating them on diverse optimization problems. More specifically, the generated algorithms are evaluated on 24 noiseless functions in a $5d$ search space using the BBOB suite [5] and using 3 instances and 3 independent runs for each of the benchmark functions (typical usage of the BBOB suite). Key challenges include navigating multimodal, separable, and highly-conditioned landscapes to achieve competitive performance within the evaluation budget (of 10 000 function evaluations). For additional details on the exact setup of this task, we refer the reader to [18].

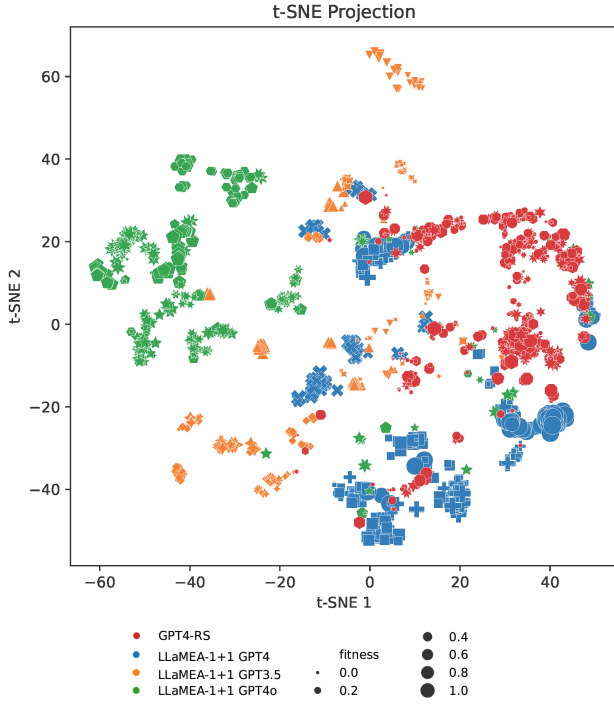


Figure 1: t-SNE visualisation of the 26 code features for different LLaMEA configurations, 5 independent runs per configuration and 5 Random Search runs. The color denotes the method, different shapes denote different independent runs and different sizes denote different normalized fitness (bigger is better).

Figure 1 shows the t-SNE projection of the extracted 26 code features for different LLaMEA configurations and Random Search (RS) runs. Each marker represents a single optimization algorithm, with different shapes denoting independent LLaMEA runs and marker sizes corresponding to fitness values (larger markers indicate higher fitness). The visualization reveals several insights into the behavior of the optimization process.

First, the runs are mostly non-overlapping, indicating distinct optimization trajectories for each configuration. Notably, only the RS experiments show some degree of overlap, suggesting that RS lacks the structured exploration seen in the LLaMEA-based methods. Second, the solutions generated by different LLMs occupy distinct regions of the feature space, highlighting unique coding “fingerprints” characteristics of each LLM. This suggests that each LLM employs unique generation and refinement strategies, producing algorithmic solutions with different structural properties. Third, the clustering of points within each optimization run suggest an “evolutionary” trend in the feature space. This pattern implies that the evolutionary loop and in-context learning mechanisms in LLaMEA effectively guide the search towards better solutions. Finally, the diversity in coding fingerprints across LLMs supports the argument for leveraging multiple LLMs rather than relying on a single model. By utilizing multiple LLMs, one can exploit their

complementary strengths, increasing the likelihood of generating diverse and high-quality solutions.

Figure 2 presents the Code Evolution Graphs for the BBO benchmark. The left side visualizes the CEGs using the first principal component on the 22 AST graph code features (without complexity features) as the y -axis, while the right side depicts the total token count on the y -axis to show an indicator of total code-complexity. The first PCA component coefficients can be found in our Zenodo repository [19], indicating which features explain most variability in the data. Each row corresponds to a different AAD algorithm (from top to bottom; LLaMEA using GPT-4-Turbo, GPT-4o, GPT3.5 and Random Search using GPT-4-Turbo). Each column shows independent optimization runs. From these visualizations, we observe distinct structural patterns in the evolution of algorithms. LLaMEA uses by default a 1 + 1 strategy with one parent generating one offspring algorithm. This results in a CEG where each node (algorithm) is connected to exactly one parent node. The graphs show how many mutations it took on a parent algorithm to finally improve the fitness. In the Random Search algorithm there is no notion of parents and offspring, and so they are not connected by edges.

4.2 Online Bin Packing

The Online Bin Packing problem is a classic combinatorial optimization challenge where items of varying sizes arrive sequentially and must be placed into a finite set of bins with fixed capacity. The objective is to minimize the total number of bins used while adhering to capacity constraints. OBP requires algorithms to make decisions in real time, without knowledge of future items.

The AAD task for OBP involves generating heuristics that can efficiently determine bin placement strategies for incoming items. These heuristics are evaluated on their ability to generalize across multiple problem instances, where item sequences vary. The evolution process must identify effective scoring functions or decision rules that minimize bin usage while maintaining computational efficiency. For additional details on the exact setup of this task, we refer the reader to [4].

The Code Evolution Graphs for OBP are shown in Figure 3. Similar to the BBO visualizations, PCA projections and code token count analyses are used to capture the structural evolution of the algorithms. The CEGs illustrate how LLaMEA-HPO and EoH adapt their strategies over successive generations. Evolution of Heuristics uses by default a 4 + 20 strategy with a population size of 4 parents, and each parent undergoing 5 different mutations to end up with 20 offspring. This shows in the CEG as a much more densely connected graph. We can also observe that EoH generates mutations that are relatively close to each other and there is no clearly visible exploration in the PCA feature space compared to LLaMEA-HPO. Note that hyper-parameter optimization in LLaMEA-HPO only affects the fitness of the algorithms and not their features.

4.3 Traveling Salesperson Problems

The Traveling Salesperson Problem is a well-known NP-hard problem in which the objective is to determine the shortest possible route that visits a set of cities exactly once and returns to the starting point. Given a complete weighted graph, the task is to minimize the total length of the tour.

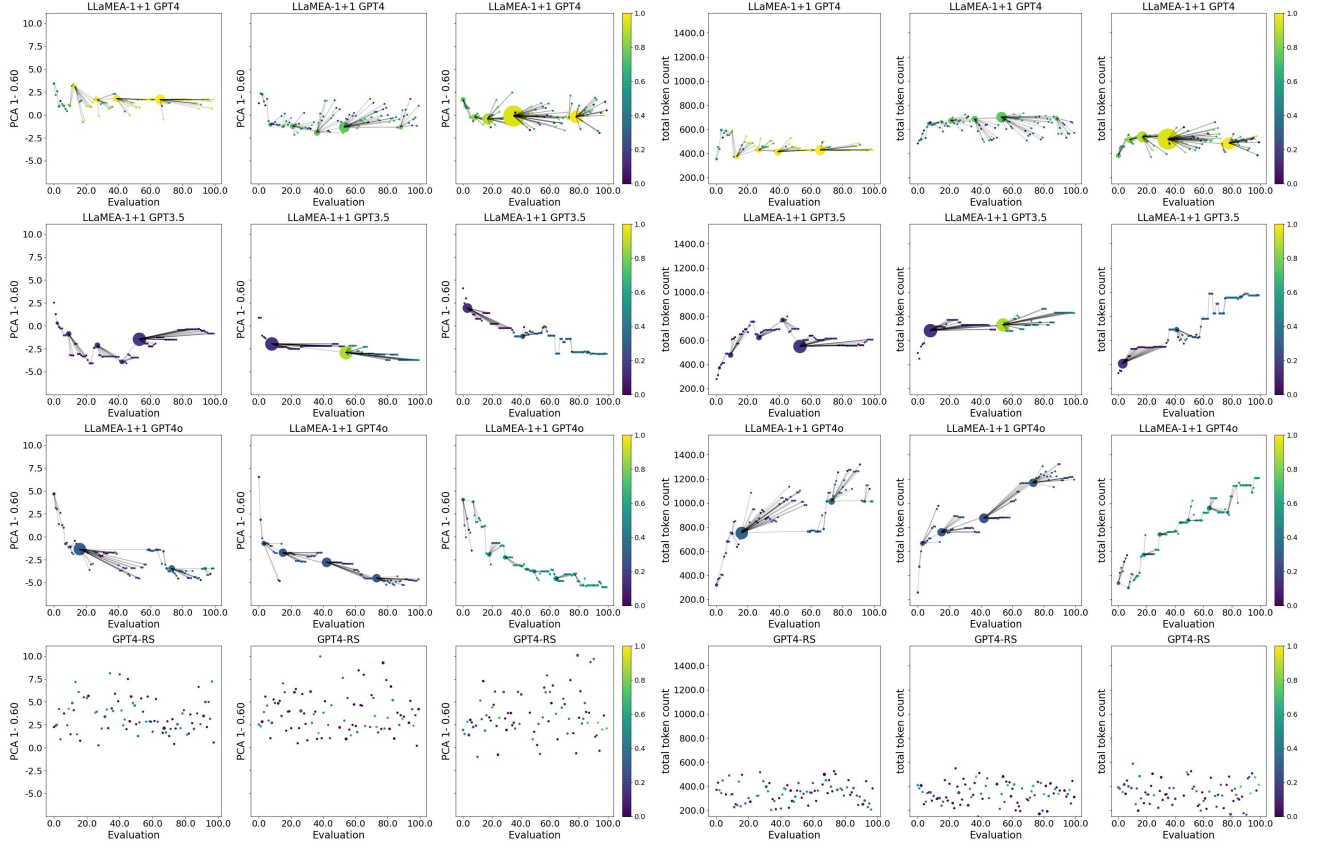


Figure 2: Code Evolution Graphs for LLaMEA using different LLMs and a baseline Random Search on BBO. On the left side are CEGs using the first PCA component of the AST graph metrics on the y -axis, with the number denoting the fraction of the total variance accounted for by this dimension. On the right side are CEGs using the total token count on the y -axis. Each row represents a different algorithm configuration (LLM) and each column is one independent run (3 runs in total).

In the AAD setting, the challenge lies in generating heuristics that guide a local search algorithm, Guided Local Search (GLS), to explore the solution space effectively. The heuristics evolve strategies for dynamically penalizing frequently used edges, thereby encouraging exploration of alternative paths and improving convergence to near-optimal solutions. TSP serves as a benchmark for assessing the scalability and adaptability of generated algorithms across problem sizes and graph structures. For additional details on the exact setup of this task, we refer the reader to [4].

Figure 4 shows the Code Evolution Graphs for the TSP benchmark. Also in this case the EoH approach uses a population size of 4 with 20 offspring being generated each iteration. The CEGs from LLaMEA-HPO show that a near-optimal solution was found very early in the search process (in the first 20 evaluations) and that the search afterwards did not improve anymore. For EoH the search is less successful in finding a near-optimal solution very fast. We observe a clear trend of code complexity increasing with the number of evaluations.

4.4 Analysis of Code Features

To quantify how features affect the fitness of the generated algorithms, we compute the Spearman correlation between extracted code features and fitness values across all benchmark tasks and LLMs. The result is shown in Figure 5. In the BBO benchmark (top four rows), mean complexity, total parameter count, and mean parameter count emerge as the most influential, suggesting that algorithm modularity, size and logical complexity are key drivers of performance of the generated algorithms in continuous optimization tasks. The positive correlation for all those features suggests that more complex algorithms perform better. For the OBP benchmark (rows five and six), the structural properties max and mean depth play the most prominent roles, with a positive correlation suggesting again that more complex code tends to perform better. Finally, in the TSP benchmark (last two rows), max degree and mean complexity have a substantial impact. However, in contrast to the other two benchmarks, the correlation is negative, showing that less complex solutions for this benchmark provide better performance than more complex solutions.

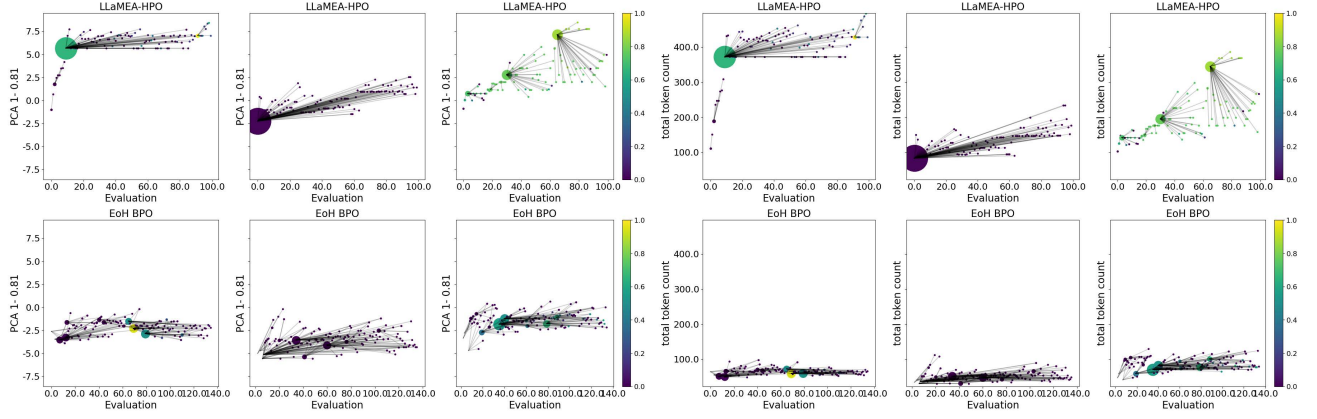


Figure 3: Code Evolution Graphs for LLaMEA-HPO and EoH on the Online Bin Packing Problems. On the left side are CEGs using the first PCA component of the AST graph metrics on the y -axis, with the number denoting the fraction of the total variance accounted for by this dimension. On the right side are CEGs using the total token count on the y -axis. The top row shows different runs of LLaMEA-HPO and the bottom row shows different runs for EoH.

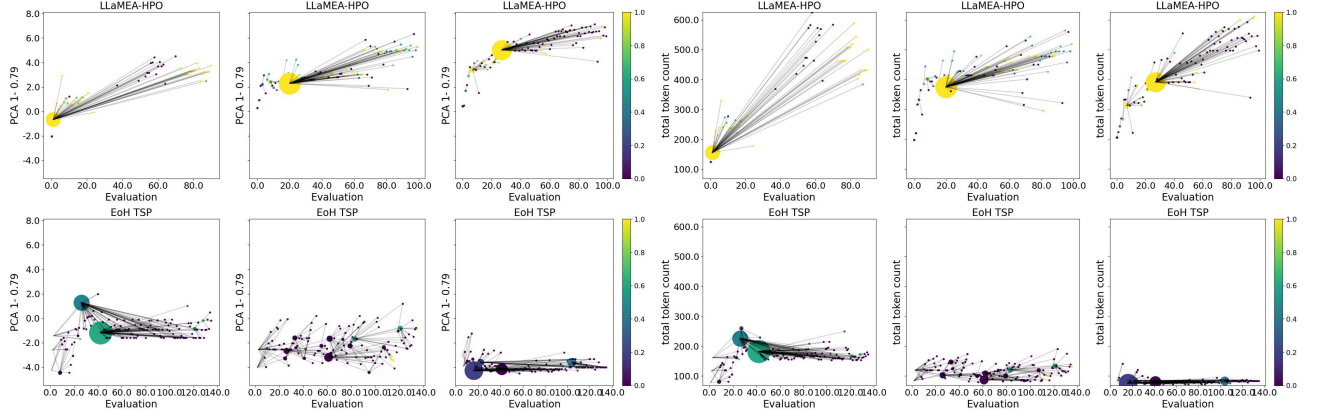


Figure 4: Code Evolution Graphs for LLaMEA-HPO and EoH on the Traveling Salesperson Problems. On the left side are CEGs using the first PCA component of the AST graph metrics on the y -axis, with the number denoting the fraction of the total variance accounted for by this dimension. On the right side are CEGs using the total token count on the y -axis. The top row shows different runs of LLaMEA-HPO and the bottom row shows different runs for EoH.

The important code features and their impact vary across benchmarks and across LLMs. There is no single set of features that always affects performance in the same way. This makes sense, as we are solving very different benchmarks with starting codes with different performances. Again we observe that different LLMs have different “fingerprints” and tend to modify code in different ways. For example, the fitness of code for BBO LLaMEA 1 + 1 GPT-4-Turbo is relatively strongly positively correlated with mean depth, while swapping GPT-4-Turbo for GPT-4o changes this to a relatively strong negative correlation. For TSP EoH, no feature shows a strong correlation with the fitness of the evolved algorithm, suggesting that changing the code does not affect performance much – the LLM is unable to find changes that improve performance. In

contrast, for BBO LLaMEA 1 + 1 GPT-3.5 and GPT-4-Turbo in particular, many features have relatively strong positive or negative correlations with performance, suggesting that changes being made to the code have a large impact on performance.

5 Conclusions and Future Work

In this paper, we introduced Code Evolution Graphs as a novel methodology for analyzing and visualizing the structural evolution of algorithms generated by LLMs in evolutionary frameworks. By extracting and analyzing a comprehensive set of structural and complexity features from Abstract Syntax Trees, we were able to uncover insights into the design principles underlying successful algorithms for different tasks. Our results demonstrated several findings:

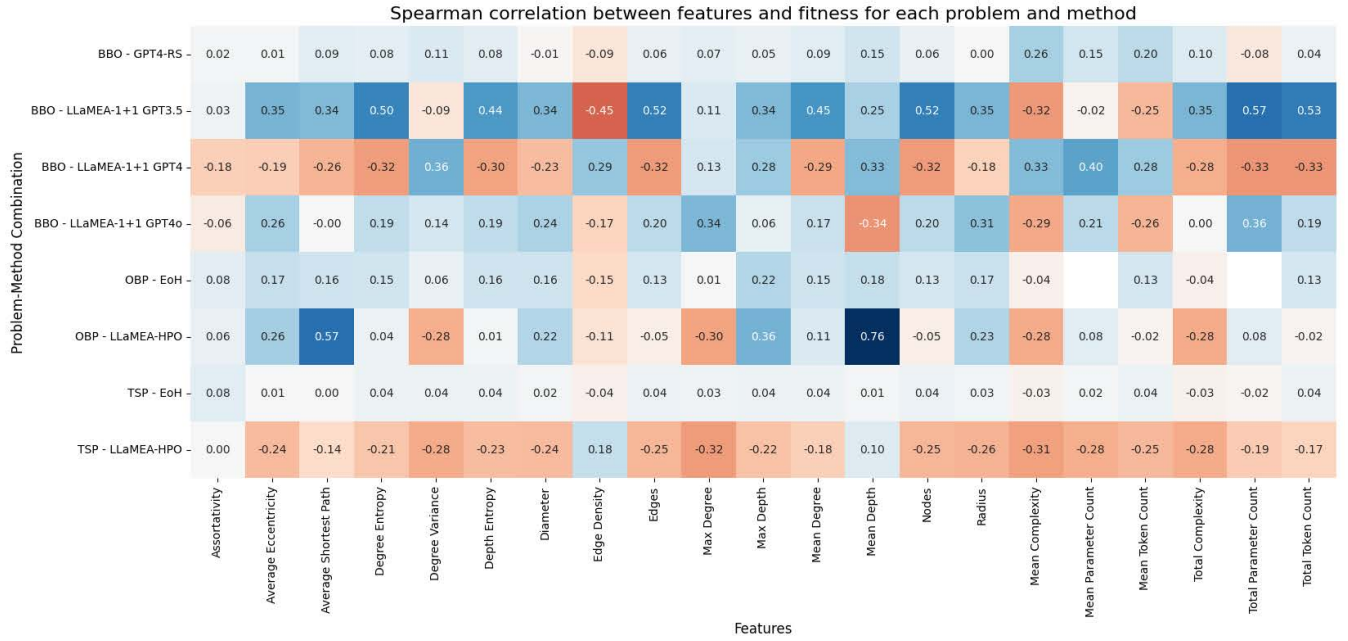


Figure 5: Spearman correlation index for each code feature (column) with the performance of the algorithms (fitness) for all benchmarks and methods (rows).

- **Diversity in Solutions:** CEGs reveal that the generated algorithms span a vast and non-overlapping feature space, emphasizing the diversity of code-space explored during the optimization process. This highlights the ability of frameworks such as LLaMEA and EoH to explore novel algorithmic structures.
- **Trends in Complexity:** We observed a consistent upward trend in code complexity (in particular, code token count) over the evolutionary process for LLaMEA-based frameworks, especially using the GPT-4o model. This suggests that the LLM-driven mutation strategies, particularly within the 1+1 evolutionary paradigm, tend to produce increasingly complex algorithms over time.
- **Task-Specific Feature Importance:** Our analysis of the correlation of feature values with fitness showed that the importance of individual features varies across benchmarks. For the Black-Box Optimization and Online Bin Packing tasks, higher complexity correlated positively with fitness. In contrast, for the Traveling Salesperson Problem, simpler solutions with lower complexity performed better, suggesting that more complex algorithms are not always better.
- **Framework Comparisons:** LLaMEA-HPO, with its in-the-loop hyper-parameter optimization, exhibited smoother evolutionary trajectories, compared to EoH's reliance on structural diversity through mutations.

These insights demonstrate the utility of Code Evolution Graphs as a tool for linking the structure and complexity of algorithms to their fitness performance. By providing a systematic approach to analyzing code evolution, CEGs enable a better understanding

of the strengths and limitations of LLM-based frameworks in the AAD domain.

While this work provides a foundation for analyzing LLM AAD frameworks, several avenues for future research remain:

- **Code Features:** One of the main limitations of the representation by AST features is that a badly configured algorithm would have exactly the same AST features as a hyper-parameter optimized algorithm. Research into other (dynamic) features would strengthen the methodology.
- **Complexity Control:** Techniques to limit or regulate code complexity during the evolutionary process could improve the efficiency and interpretability of generated algorithms.
- **Diversity Management:** Strategies for controlling diversity in the population, such as diversity-aware mutation operators, could further enhance exploration behavior.

This work bridges the gap between algorithmic evolution and code structure analysis, offering a framework for visualizing and analyzing LLM-driven algorithm design tasks. All code and additional results are available in our Zenodo repository [19].

Acknowledgments

This work has benefitted from Dagstuhl Seminar 24282, “Automated Machine Learning for Computational Mechanics”. We thank Grace Abawe for writing the original AST analysis code.

References

- [1] Rasmus Bro and Age K Smilde. 2014. Principal component analysis. *Analytical methods* 6, 9 (2014), 2812–2831.
- [2] Matthew C Cieslak, Ann M Castelfranco, Vittoria Roncalli, Petra H Lenz, and Daniel K Hartline. 2020. t-Distributed Stochastic Neighbor Embedding (t-SNE):

- A tool for eco-physiological transcriptomic analysis. *Marine genomics* 51 (2020), 100723.
- [3] Christof Ebert, James Cain, Giuliano Antoniol, Steve Counsell, and Phillip Laplante. 2016. Cyclomatic complexity. *IEEE Software* 33, 6 (2016), 27–29.
 - [4] Liu Fei, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2401.02051>
 - [5] Nikolaus Hansen, Steffen Finck, Raymond Ros, and Anne Auger. 2009. *Real-Parameter Black-Box Optimization Benchmarking 2009: Noiseless Functions Definitions*. Research Report RR-6829. INRIA. <https://hal.inria.fr/inria-00362633>
 - [6] Holger H. Hoos. 2012. Programming by Optimization. *Communications of the Association for Computing Machinery (CACM)* 55, 2 (Feb. 2012), 70–80. doi:10.1145/2076450.2076469
 - [7] Stewart Martin-Haugh, Stefan Kluth, Rolf Seuster, Scott Snyder, Emil Obreshkov, Shaun Roe, Peter Sherwood, and Graeme A Stewart. 2017. C++ software quality in the ATLAS experiment: tools and experience. In *Journal of Physics: Conference Series*, Vol. 898. IOP Publishing, 072011.
 - [8] Olaf Mersmann, Bernd Bischl, Heike Trautmann, Mike Preuss, Claus Weihs, and Günter Rudolph. 2011. Exploratory Landscape Analysis. In *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation (GECCO '11)*. Association for Computing Machinery, New York, NY, USA, 829–836. doi:10.1145/2001576.2001690
 - [9] Alberto S Nuñez-Varela, Héctor G Pérez-Gonzalez, Francisco E Martínez-Perez, and Carlos Soubervielle-Montalvo. 2017. Source code metrics: A systematic mapping study. *Journal of Systems and Software* 128 (2017), 164–197.
 - [10] Gabriela Ochoa, Katherine M. Malan, and Christian Blum. 2021. Search trajectory networks: A tool for analysing and visualising the behaviour of metaheuristics. *Applied Soft Computing* 109 (2021), 107492. doi:10.1016/j.asoc.2021.107492
 - [11] Gabriela Ochoa, Sébastien Verel, Fabio Daolio, and Marco Tomassini. 2014. Local optima networks: A new model of combinatorial fitness landscapes. *Recent advances in the theory and application of fitness landscapes* (2014), 233–262.
 - [12] OpenAI. 2023. ChatGPT-4o. <https://platform.openai.com/docs/models/gpt-4o>.
 - [13] Damir Pulatov, Marie Anastacio, Lars Kotthoff, and Holger Hoos. 2022. Opening the black box: Automated software analysis for algorithm selection. In *International Conference on Automated Machine Learning*. PMLR, 6–1.
 - [14] Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical discoveries from program search with large language models. *Nature* 625 (01 2024), 468–475. Issue 7995.
 - [15] Alexandru Telea and David Auber. 2008. Code flows: Visualizing structural evolution of source code. In *Computer Graphics Forum*, Vol. 27. Wiley Online Library, 831–838.
 - [16] Sarah L. Thomson, Quentin Renau, Diederick Vermetten, Emma Hart, Niki van Stein, and Anna V. Kononova. 2024. Stalling in Space: Attractor Analysis for any Algorithm. arXiv:2412.15848 [cs.NE] <https://arxiv.org/abs/2412.15848>
 - [17] Sander van Rijn, Hao Wang, Matthijs van Leeuwen, and Thomas Bäck. 2016. Evolving the structure of Evolution Strategies. In *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, 1–8. doi:10.1109/SSCI.2016.7850138
 - [18] Niki van Stein and Thomas Bäck. 2024. LLaMEA: A Large Language Model Evolutionary Algorithm for Automatically Generating Metaheuristics. *IEEE Transactions on Evolutionary Computation* (2024), 1–1. doi:10.1109/TEVC.2024.3497793
 - [19] Niki van Stein, Anna V. Kononova, Lars Kotthoff, and Thomas Bäck. 2025. Figures and code for Code Evolution Graphs. doi:10.5281/zenodo.14770672
 - [20] Niki van Stein, Diederick Vermetten, and Thomas Bäck. 2024. In-the-loop Hyper-Parameter Optimization for LLM-Based Automated Design of Heuristics. arXiv:2410.16309 [cs.NE] <https://arxiv.org/abs/2410.16309>
 - [21] Niki van Stein, Diederick Vermetten, Anna V. Kononova, and Thomas Bäck. 2024. Explainable Benchmarking for Iterative Optimization Heuristics. (2024). arXiv:2401.17842 [cs.NE] arXiv:2401.17842.
 - [22] Diederick Vermetten, Martin S. Krejca, Marius Lindauer, Manuel López-Ibáñez, and Katherine M. Malan. 2024. Synergizing Theory and Practice of Automated Algorithm Design for Optimization (Dagstuhl Seminar 23332). *Dagstuhl Reports* 13, 8 (2024), 46–70. doi:10.4230/DagRep.13.8.46
 - [23] Diederick Vermetten, Sander van Rijn, Thomas Bäck, and Carola Doerr. 2019. Online selection of CMA-ES variants. In *Proceedings of the Genetic and Evolutionary Computation Conference (Prague, Czech Republic) (GECCO '19)*. Association for Computing Machinery, New York, NY, USA, 951–959. doi:10.1145/3321707.3321803
 - [24] Rui Zhang, Fei Liu, Xi Lin, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Understanding the Importance of Evolutionary Search in Automated Heuristic Design with Large Language Models. In *International Conference on Parallel Problem Solving from Nature*. Springer, 185–202.